MediaMTX + FFmpeg Streaming Setup

Gujarat Police Hackathon - Practical Setup, Troubleshooting and Portable Design

Scope: This document records only the MediaMTX + FFmpeg streaming work we performed, plus the Docker pieces needed to make MediaMTX portable. It intentionally does not document the complete Docker/Compose system.

1. What We Were Trying to Build

The objective was to take a real camera RTSP stream, place MediaMTX between the camera and the application, and make the stream available to browser-friendly protocols. FFmpeg was also used as a temporary fake camera source during testing by repeatedly encoding a PNG image into H.264/RTSP.

```
Real camera
    |
    | RTSP
    v
MediaMTX
    |-----------------> RTSP client (VLC)
    |-----------------> HLS / WebRTC
    |
    +---- Control API ----> FastAPI backend
```

The final real-camera path that we successfully tested was **camera -> MediaMTX -> browser**. The browser opened the MediaMTX HLS page for the camera stream.

2. Technologies Used in This Streaming Stage

Technology	Purpose in this work
MediaMTX 1.21.0	RTSP server/proxy and restreaming layer; also exposed HLS, WebRTC and a Control API.
FFmpeg	Temporary test-source generator and RTSP publisher. It converted a still PNG into H.264 and published an RTSP stream.
ffplay / VLC	Direct camera test and RTSP playback verification.
YAML	MediaMTX configuration file format.
Docker	Used to run a Linux MediaMTX binary in a portable container rather than depending on the Windows executable.

3. MediaMTX Folder We Started From

```
streaming/
■■■ mediamtx_v1.21.0_windows_amd64/
    ■■■ auto.crt
    ■■■ auto.key
    ■■■ LICENSE
    ■■■ mediamtx.exe
    ■■■ mediamtx.yml
```

The working Windows installation was kept as a backup/reference. The Windows **mediamtx.exe** was not copied into the Linux Docker image. This is important for portability.

4. MediaMTX Configuration Concepts We Used

The working configuration included RTSP, HLS and WebRTC listeners, internal authentication and the Control API. During the final setup the API was enabled so FastAPI could create/query MediaMTX paths.

```
api: true
apiAddress: :9997

# examples of listeners observed in MediaMTX logs
RTSP:    :8554
HLS:     :8888
WebRTC:  :8889
ICE/UDP: :8189
```

Important distinction: port 8888 was HLS in our setup; port 8889 was WebRTC. Port 9997 was the Control API. Port 8554 was RTSP.

5. Dockerfile Used to Make MediaMTX Portable

Instead of committing a Windows executable, the Docker image downloads the Linux AMD64 MediaMTX release and then copies the project configuration file into the image.

```
FROM alpine:3.20

ARG MTX_VERSION=1.21.0

RUN apk add --no-cache wget ca-certificates && \
    wget -q https://github.com/bluenviron/mediamtx/releases/download/v${MTX_VERSION}/mediamtx_v${MTX_VERSION}_linux_amd64.tar
    tar -xzf mediamtx_v${MTX_VERSION}_linux_amd64.tar.gz && \
    mv mediamtx /usr/local/bin/mediamtx && \
    rm mediamtx_v${MTX_VERSION}_linux_amd64.tar.gz

COPY streaming/mediamtx_v1.21.0_windows_amd64/mediamtx.yml /mediamtx.yml

EXPOSE 8554/tcp
EXPOSE 8000/udp
EXPOSE 8001/udp
EXPOSE 1935/tcp
EXPOSE 8888/tcp
EXPOSE 8889/tcp
EXPOSE 8189/udp
EXPOSE 8890/udp
EXPOSE 8892/tcp
EXPOSE 8892/udp
EXPOSE 8893/udp

CMD ["/usr/local/bin/mediamtx", "/mediamtx.yml"]
```

Why this is portable: the container gets the Linux MediaMTX binary itself. A second developer does not need the Windows **mediamtx.exe** to run the container. The project configuration remains a normal text file.

6. Why COPY Shows mediamtx.yml Twice

```
COPY streaming/mediamtx_v1.21.0_windows_amd64/mediamtx.yml /mediamtx.yml
```

The first path is the **source on the Docker build context**. The second path is the **destination inside the container**. They have the same filename by design, but they are two different locations.

7. FFmpeg Test Camera We Used

For initial testing we used a PNG as a fake camera. FFmpeg repeatedly read the image, converted it to H.264 and published it to the MediaMTX RTSP path **test**.

```
& "C:\...\ffmpeg.exe" `
-loop 1 `
-framerate 25 `
-i "C:\Users\HP\OneDrive\Pictures\test.PNG" `
-vf "scale=500:282" `
-c:v libx264 `
-preset veryfast `
-tune stillimage `
-pix_fmt yuv420p `
-r 25 `
-fps_mode cfr `
-f rtsp `
-rtsp_transport tcp `
rtsp://127.0.0.1:8554/test
```

The image was 500x483. We scaled to 500x282 so the encoded video dimensions were even and suitable for yuv420p/H.264.

The same FFmpeg executable was used only as a temporary local test tool. The final real-camera path did not require FFmpeg because the real camera already produced RTSP.

8. How We Verified the Test Stream

```
RTSP source:
rtsp://127.0.0.1:8554/test

VLC:
Open network stream -> rtsp://127.0.0.1:8554/test
```

MediaMTX logs confirmed both roles: FFmpeg was **publishing to path 'test'**, and VLC was **reading from path 'test'**. The image was visible in VLC, proving the RTSP publish/read path worked.

9. Problems We Faced and How We Solved Them

FFmpeg loop option rejected

Problem: The command used **-loop 2** for the image input. This FFmpeg image demuxer expects a boolean, so it reported: **Unable to parse "loop" option value "2" as boolean**.

Solution: Changed it to **-loop 1**.

Result: FFmpeg accepted the input and started publishing.

FFmpeg -> MediaMTX ended with broken pipe

Problem: FFmpeg reported **Failed reading RTSP data: End of file** followed by **Broken pipe**. The test source was synthetic and MediaMTX was losing the publisher session.

Solution: We changed the test to use explicit real-time input/output settings and then used TCP RTSP transport for the connection.

Result: The later test stayed connected long enough to be read by VLC, and the RTSP path worked.

VLC initially could not open /test

Problem: MediaMTX logs showed that the publisher disappeared, so there was temporarily no active stream for VLC to read. VLC also attempted invalid/unsupported RTSP requests during retries.

Solution: We inspected MediaMTX logs instead of treating the VLC message alone as the root cause.

Result: The logs later showed a valid reader session and the image became visible in VLC.

MediaMTX API unreachable from Windows

Problem: The API was configured at :9997 but the container did not initially publish port 9997 to the host.

Solution: Added host-to-container port mapping **9997:9997**.

Result: PowerShell could reach the API endpoint.

MediaMTX API authentication failure

Problem: The API returned **authentication error**. The existing 'any' user with IP 127.0.0.1 only represented localhost inside the container.

Solution: Created a development API user with **api** permission and tested with HTTP Basic Authentication.

Result: The API returned HTTP 200 with a valid JSON path list.

RTSP source used literal CAMERA_IP

Problem: MediaMTX tried to resolve the hostname **CAMERA_IP** and returned a Docker DNS error.

Solution: Replaced the placeholder with the real camera host/RTSP URL.

Result: The real camera became ready and available.

YAML source had an extra colon

Problem: A source line ended with an extra colon, causing YAML parsing to fail.

Solution: Removed the extra colon and kept the RTSP URL as a quoted YAML scalar.

Result: MediaMTX loaded the configuration.

Vendor URL contained literal quote characters

Problem: The database contained a base URL such as **"rtsp://.../cam"**. Appending **03** generated **"rtsp://.../cam"03**, which MediaMTX rejected.

Solution: Store the VARCHAR URL without surrounding quote characters and defensively use **vendor[2].strip().strip('"')** before generating camera URLs.

Result: Generated URLs became **.../cam00**, **.../cam01**, etc.

10. Real Camera Integration

After the fake FFmpeg source was verified, we switched to the real RTSP camera. MediaMTX was configured to pull the camera source instead of receiving a stream generated by FFmpeg.

```
paths:
  cam01:
    source: "rtsp://USERNAME:PASSWORD@CAMERA_HOST:8554/stream/cam01"
```

The real camera was confirmed with the direct ffplay test first. Then MediaMTX reported **ready=true**, **available=true** and an **H264** track. The stream was then opened in the browser through the MediaMTX HLS page.

```
Browser HLS test:
http://127.0.0.1:8888/cam01/
```

11. Multi-Camera Design

The original application does not need one static MediaMTX configuration entry for every camera. The database stores a vendor base URL and the application generates individual CameraURL values using the camera number.

```
BASE_URL = vendor[2].strip().strip('"')

for i in range(vendor[1]):
    cam_url = f"{BASE_URL}{i:02d}"
    camera_name.append(f"cam{i:02d}")
```

The FastAPI stream-start function then reads **CameraURL** from MySQL and dynamically adds a MediaMTX path through the Control API. This is the foundation for handling many cameras without writing every camera path manually into **mediamtx.yml**.

12. FastAPI -> MediaMTX Connection

```
MEDIAMTX_API = "http://gujarat_police_mediamtx:9997"
MEDIAMTX_WEBRTC = "http://127.0.0.1:8889"
```

The first address is for **backend-to-MediaMTX** communication inside Docker. The second is a browser-facing host address in the current local setup. This distinction matters because **127.0.0.1 inside the backend container is not the MediaMTX container**.

```
POST /v3/config/paths/add/{stream_name}

{
  "source": camera_url,
  "rtspTransport": "tcp"
}
```

13. Portable Version - What We Keep and What We Do Not

The portable goal is to keep configuration and code in the repository while avoiding machine-specific Windows executables.

Keep in repository	Keep local / obtain at build time
mediamtx.yml	Windows mediamtx.exe
deployment/docker/mediamtx.Dockerfile	Local FFmpeg/ffplay/ffprobe .exe files
FastAPI stream-control code	Developer-specific camera credentials
Docker Compose service definition	Local cache/runtime files

For FFmpeg, the local Windows binaries were deliberately excluded from Git because they are very large. For a fully portable future deployment, FFmpeg should be supplied through a container/image or installed by an explicit setup step rather than bundled as Windows executables.

Security note: RTSP credentials should not be committed to the repository. Use environment variables/secrets for deployment. The real credential used during testing is intentionally not reproduced in this document.

14. Useful Verification Commands

```
# Show MediaMTX container and published ports
docker ps

# Read recent MediaMTX logs
docker logs gujarat_police_mediamtx --tail 50

# Check MediaMTX API from the Windows host
Invoke-WebRequest `
  -Uri "http://127.0.0.1:9997/v3/paths/list" `
  -Headers @{ Authorization = "Basic <BASE64>" } `
  -UseBasicParsing

# Verify backend can reach MediaMTX from inside Docker
docker exec gujarat_police_backend python -c "import httpx; r=httpx.get('http://gujarat_police_mediamtx:9997/v3/paths/list',

# Verify an RTSP listener from Windows
Test-NetConnection 127.0.0.1 -Port 8554
```

15. Final Working Streaming Path

```
REAL CAMERA
     |
     | RTSP / H264
     v
+----------------------+
| MediaMTX Docker      |
| :8554 RTSP           |
| :8888 HLS            |
| :8889 WebRTC         |
| :9997 Control API    |
+----------+-----------+
           |
           +----> Browser (HLS/WebRTC)
           |
           +<---- FastAPI (Control API)
```

This setup proved the real camera can be received by MediaMTX and viewed through the browser, while FastAPI can communicate with MediaMTX through its Docker network address.

16. Next Portable Streaming Step

The next logical improvement is to containerize the FFmpeg side as well. That would remove the remaining dependency on the Windows FFmpeg executable. The real-camera case may not need FFmpeg at all when the camera already provides a suitable H264 RTSP stream; FFmpeg is still useful when transcoding, resizing, recording, or converting camera protocols is required.